

Morse Code Converter

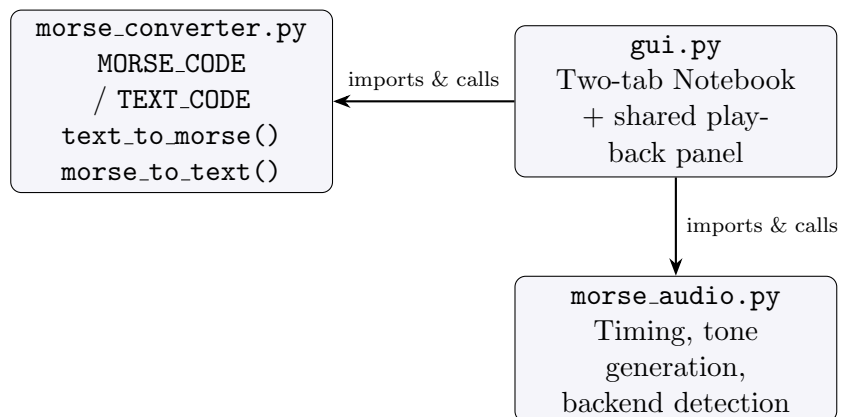
Brief

Explainer document for the owner

1 What it is

A three-file Python project: `morse_converter.py` holds pure conversion logic (no UI), `morse_audio.py` generates real audio tones and detects how to play them (also no UI), and `gui.py` is a Tkinter (Python's built-in desktop GUI toolkit) window that wraps both in a live, two-way converter with a playback panel that flashes dots/dashes on a canvas *and*, new in this version, plays them as real tones in sync.

2 Architecture at a glance



`gui.py` imports the two conversion functions and several audio functions; it never touches the Morse lookup dictionaries or the audio internals (sine-wave math, file I/O) directly. This separation means both the conversion logic and the audio generation can be tested with no Tkinter dependency at all.

3 Conversion logic

`MORSE_CODE` maps every supported character (A–Z, 0–9, space, and common punctuation) to a string of dots and dashes; `TEXT_CODE` is the same mapping built in reverse with a one-line dict comprehension. The space character maps to `" / "`, the conventional Morse word separator.

`text_to_morse(text)` uppercases the input, looks up each character, and joins the patterns with single spaces (a literal input space becomes the three characters `" / "`). An unmapped character raises `ValueError` naming it, instead of a bare `KeyError`.

`morse_to_text(morse)` reverses this: it splits on the exact substring " / " to recover words, splits each word on whitespace to recover letters, looks each pattern up in `TEXT_CODE`, and raises `ValueError` on an unmapped pattern. Splitting on the exact three-character separator (rather than a bare "/") is what keeps the round trip clean with no stray spaces.

Running `morse_converter.py` directly still gives the original one-line CLI prompt; `gui.py` is an additional interface, not a replacement.

4 The GUI

A `ttk.Notebook` (a themed tab bar) holds two tabs built from one shared helper function, differing only in which conversion function and labels they use: “Text to Morse” and “Morse to Text”. Each tab has an input `Text` box, an inline error label, a disabled (read-only) output `Text` box, and Copy/Clear buttons.

Every tab’s input box is bound to `<KeyRelease>`: on every keystroke, the handler reads the full input, calls the tab’s conversion function, and either fills the output box or shows the `ValueError` message in the error label. This runs even on invalid partial input (e.g. mid-typed Morse), which is exactly why the error is caught here rather than crashing the app.

Copy uses Tkinter’s built-in clipboard calls (no dependency); Clear resets both boxes and the error label for that tab.

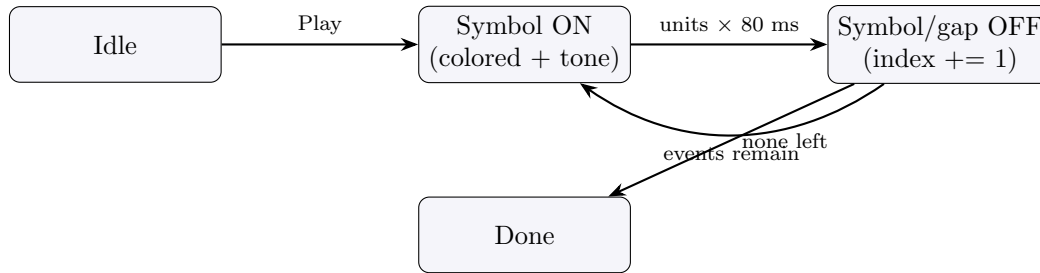
5 Playback panel

Below the tabs sits one shared `Canvas` and one Play button, used by both tabs rather than duplicated per tab. Pressing Play first decides *which* Morse string to animate: the Text-to-Morse tab’s output is checked first (it is tagged “`output`” as its Morse source); only if that is empty does it fall back to the Morse-to-Text tab’s input (tagged “`input`”, since that tab’s input box is the one holding Morse). This is a fixed priority order, not “whichever was edited last.”

The chosen string is turned into a list of (`kind`, `units`) timing events by `build_timing_sequence()` (from `morse_audio.py`, detailed in §6), then one shape per dot/dash is drawn left to right (small ovals for dots, wider rectangles for dashes); gaps get no shape but still occupy real time and extra horizontal space, so letter and word boundaries are visible.

5.1 Timing state machine

Animation advances via `root.after(ms, callback)`, Tkinter’s non-blocking “run this once, later” scheduler — there is no separate thread. Each step schedules exactly one future step, so the chain is *recursive*, not a loop, and it now drives both the flash and the new audio tone off the same event list:

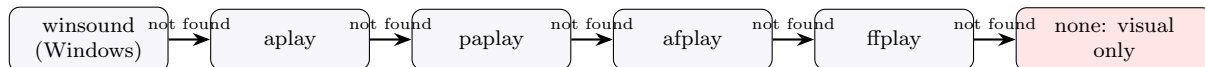


Each event’s on-screen duration is its unit count times 80 ms (UNIT_MS), following the real Morse ratios (dot 1 unit, dash 3 units, gap within a letter 1, between letters 3, between words 7) rather than the flat approximate timing an earlier version used. Pressing Play again cancels any in-flight animation first, so repeated presses cannot stack.

6 Audio playback

New in this version: the Play button plays real audio tones (a 700 Hz sine wave), not just a visual flash, generated entirely from the Python standard library (the `wave` module for the WAV file, `math.sin` for the tone itself, no third-party audio package).

At startup, the app calls `detect_playback_backend()` once to find out, at runtime, what can actually play a sound on this machine — never assuming a platform’s usual tool is present. It checks, in order: `winsound` (Windows only) first, then whichever of `aplay`, `paplay`, `afplay`, `ffplay` is found on the system PATH via `shutil.which`:



If a backend is found, the app pre-generates two short WAV files (one dot tone, one dash tone) once, in a temporary directory cleaned up automatically on exit, and reuses them for every symbol played thereafter. If nothing is found, `gui.py` shows a status line reading exactly “No audio output available on this system, showing visual playback only” and the visual flash keeps working unchanged — no crash, no silent no-op.

Playback itself is **fire-and-forget**: each tone is started with `subprocess.Popen` (or `winsound.PlaySound` with `SND_ASYNC` on Windows), which launches the player and returns immediately rather than waiting for it to finish. This matters because Tkinter’s own `root.after` timer drives the visual flash on a fixed schedule; if playback blocked until each tone finished, the flash would lag behind the sound by however long that tone took to play. Because starting playback never blocks, both the flash and the tone read off the same `build_timing_sequence()` event list and start each symbol at the same moment.

What was actually verified: in the Linux container this project was built in, both `aplay` and `ffplay` were present (`paplay/afplay` were not); `aplay` was selected (earliest match) and confirmed to actually play a rendered WAV (subprocess exit code 0), and a rendered “SOS” WAV’s duration matched the timing formula’s prediction exactly (2160 ms). The `winsound` and `afplay` code paths exist and use documented APIs but were not run on Windows or macOS — only the Linux/`aplay` path was verified end to end, matching the README’s own wording.

7 Deliberately out of scope

No persistence, no network calls, no configuration file, and no automated unit tests in the repository. On the audio side specifically: no real cross-platform audio device API (the app shells out to whatever command-line player the OS has, rather than writing to a sound device directly), so there is a small per-symbol process-spawn delay and no way to confirm a tone actually finished playing — it is timed against the visual clock, not the player subprocess’s own lifecycle.

8 Glossary

- **Tkinter** / **ttk** — Python’s built-in GUI toolkit; **ttk** is its themed widget set, used here for tabs, buttons, and frames.
- **Dict comprehension** — a compact Python expression that builds a dictionary from a loop in one line.
- **ValueError** — a built-in exception raised here to signal an invalid character or Morse symbol, instead of a raw **KeyError**.
- **root.after(ms, fn)** — schedules **fn** to run once after **ms** milliseconds without blocking the GUI’s event loop.
- **WAV** / **PCM** — WAV is a simple uncompressed audio file format; PCM is the raw-sample representation it stores (one number per tiny time slice). Python’s stdlib **wave** module reads and writes it.
- **shutil.which** — stdlib function that searches the system **PATH** for a command by name, the same lookup a shell does.
- **Fire-and-forget** / **non-blocking** — starting an operation (here, an external audio player process) and moving on immediately, rather than waiting for it to finish.